Mini Lesson: TCP | HTTP | Websocket

(https://github.com/learn-co-curriculum/python-p4-tcp-http-and-websocket) (https://github.com/learn-co-curriculum/python-p4-tcp-http-and-websocket/issues/new)

Learning Goals

- Compare TCP, HTTP, and Websocket in the context of full-stack web development.

Key Vocab

- **Full-Stack Development**: development of a frontend and a backend for an application. True full-stack development includes a database, a logic/server layer, and a frontend built in JavaScript, HTML, and CSS.
- **Backend**: the layer of a full-stack application that handles business logic and other programmatic tasks that users do not or should not see. Can be written in many languages, including Python, Java, Ruby, PHP, and more.
- **Frontend**: the layer of a full-stack application that users see and interact with. It is always written in the frontend languages: JavaScript, HTML, and CSS. (*There are others now, but they are based on these three.*)
- **Cross-Origin Resource Sharing (CORS)**: a method for a server to indicate any ports (or other identifiers) for servers that can share its resources.
- **Transmission Control Protocol (TCP)**: a protocol that defines how computers send data to each other. A connection is formed and stars active until the applications on either end have finished sending data to one another.
- **Hypertext Transfer Protocol (HTTP)**: a stateless protocol where applications communicate for the length of time that it takes for data to be transferred.
- **WebSocket**: a protocol that allows clients and servers to communicate with one another in both directions. The bidirectional nature of websocket communication allows a connected state to be generated and the connection maintained until it is terminated by one side. This allows for speedy and seamless connections between frontends and backends.

Introduction

Through this whole unit, we have seen information transmitted from servers to clients. We have only done this fairly short-distance, but with Procfiles and deployment platforms such as Render and Heroku, we can use the same code to communicate across the globe through the internet.

Communication between so many internet-connected devices across such a wide area requires a certain set of standards. All internet communication relies on the Internet Protocol (IP) and Transmission Control Protocol (TCP). IP dictates how messages are sent between networks on the internet. It is upheld by nearly all internet-based applications.

In this lesson, we will focus on TCP, HTTP, and WebSocket, three communication protocols that vary a bit in usage between web applications.

TCP

Where IP is focused on messages, TCP is focused on connections. It is a stateful protocol, meaning that it remembers previous transactions for the duration of the connection.

Imagine you want to send a message to your friend who lives far away. You decide to write a letter and send it through the mail. Now, to ensure that your friend receives the message correctly, you want to make sure it doesn't get lost or mixed up along the way. That's where TCP comes in.

TCP is like a smart postal service for sending data over the internet. It takes your message and breaks it down into smaller pieces called packets. Each packet contains a part of your message and some additional information.

Next, TCP adds some special information to each packet. It includes the address of the sender (that's you) and the address of the receiver (your friend). This way, the packets know where to go.

TCP also numbers each packet in order. It's like labeling the packets with numbers, starting from one, so your friend can put them back together correctly. This is important because the packets may not arrive in the same order they were sent. They can take different routes over the internet, and sometimes they arrive out of order.

Now, your packets are ready to be sent. TCP takes place in the **transport layer** of the networking stack. The transport layer is like the post office, handling the transportation of your letter. It takes care of how the letter will be delivered from your location to your friend's location. It

ensures that the letter is sent reliably and efficiently, making sure it doesn't get lost or damaged along the way. The transport layer uses different methods, just like different shipping companies might use trucks, airplanes, or ships to transport packages. TCP hands them over to the internet. The internet ensures that the packets are delivered to your friend's computer.

When your friend's computer receives the packets, TCP comes into action again. It checks if any packets are missing or got damaged during the journey. If that happens, TCP asks the sender to send those missing or damaged packets again. This ensures that your friend gets all the packets and can read your message correctly.

Finally, TCP puts the packets back together in the correct order. It looks at the numbers on the packets and arranges them in the right sequence. Once all the packets are in the right order, your friend's computer can read your complete message.

TCP is considered a stateful protocol because it establishes a connection, keeps track of the ongoing communication, ensures packets arrive reliably and in order, manages flow control, and maintains the connection until the conversation is complete. This statefulness allows for efficient and error-free data transmission over the internet.

So, TCP is like a smart postal service that breaks your message into smaller pieces, labels them with numbers, sends them over the internet, and ensures that they arrive in the right order and without any errors. It makes sure your messages reach their destination *reliably* and *correctly*.

Hypertext Transfer Protocol (HTTP)

Hypertext Transfer Protocol, or HTTP, is built atop TCP, sending requests and receiving responses atop an underlying TCP connection. HTTP allows users and applications to communicate via hypertext (e.g., HTML). It's the foundation of the World Wide Web and is widely used for accessing internet resources.

HTTP follows a client-server model, where the client is usually a web browser, and the server is, well...a server! Every web application has its own server to handle the requests and deliver the requested resources.

When using HTTP, the client initiates a connection with the server to make a request. It sends a message to the server asking for a specific resource, such as a webpage or a file. The client then waits for the server's response. Once the response is received, the connection is closed. This means that HTTP is stateless, even though it is built on top of TCP, which is a stateful protocol.

Being stateless means that the server doesn't remember any information about the client's previous requests. It treats each request as an independent, standalone request. This design simplifies server implementation, improves scalability, and allows for better load balancing.

HTTP uses the **application layer** of the networking stack. Returning to our letter-carrier analogy, application layer is like the content of the letter. It's the actual reason why you are sending the letter in the first place. It could be a birthday gift, a letter, or a book. The application layer determines what type of information or resource you want to send to your friend. The application layer is *on top of the transport layer* in the networking stack- this means that HTTP requests are sent after work in the transport layer is complete.

In HTTP, requests are typically sent over port 80. When you enter a URL in your browser without specifying a port, it defaults to port 80. However, if you see a lock icon to the left of the domain name in your browser, it indicates that you're using port 443 for HTTPS.

HTTPS

Hypertext Transfer Protocol Secure (HTTPS) uses encryption to ensure secure communications between the client and server. The protocol used for encryption is called Transport Layer Security (TLS) and secures communications with a public key and a private key.

- A **public key** is available to anyone who wanted to interact with a server over HTTPS. The public key is used to encrypt a website's data.
- A **private key** is only available to the owner of a website. It lives on the server and decrypts information that has been encrypted by the public key.

All communication over HTTP uses easily-readable plain text, which does not leave your data very secure on its own. Encryption ensures that others in your network cannot see the plain text that you're sending and receiving when using the internet.

Summary: TCP vs. HTTP

1. Purpose

- TCP is responsible for reliable data transmission, ensuring packets are delivered without errors and in the correct order.
- HTTP is used for communication between web clients (such as web browsers) and servers. It defines how clients request resources, and servers respond with those resources.

1. Layer:

- TCP operates at the transport layer of the networking stack, providing a reliable and ordered delivery of data over the internet.
- HTTP operates at the application layer, which is higher in the networking stack compared to TCP.

1. Connection:

- TCP is stateful. It establishes a connection between a sender and a receiver before data transmission. It ensures a reliable and ordered exchange of data by using acknowledgments and retransmission of lost packets.
- HTTP is stateless and doesn't maintain a persistent connection between the client and server. It uses a request-response model, where the client sends a request to the server, and the server responds with the requested resource. Each request-response cycle is independent.

1. Reliability:

- TCP provides reliable data transmission by using acknowledgments, error-checking, and retransmission of lost packets. It guarantees that data arrives at the destination without errors and in the correct order.
- Being stateless, HTTP does not provide any of the same reliability measures as TCP. HTTP depends on its underlying TCP connection for reliability.

WebSocket

Full-stack applications often desire fast, real-time updates between the client and server. Updates to HTTP have improved its ability to serve as a client-server intermediary in this context, but it's still far from perfect. To improve communication in a full-stack context, a team of engineers designed a new protocol: **WebSocket**.

Like HTTP, WebSocket is built on top of an existing TCP connection. Unlike HTTP, the connection is bidirectional and stateful: the client and server remain connected and can communicate with each other freely without establishing new connections.

WebSocket starts as an HTTP request: the client sends a request to the server, though this request is to open a WebSocket connection. If this request is successful, the TCP connection between client and server is reinterpreted as a WebSocket connection. Data can continue traveling back and forth until one side terminates the connection.

Traditional TCP connections require developers to write their own transfer protocols. While this allows for customization on the developer's part- which is necessary in some niche circumstances- by and large, it makes working with pure TCP slower and more difficult.

WebSocket solves this problem by establishing TCP connections with its own standardized, high-level communication protocol. This allows developers to set up WebSockets without having to configure a TCP protocol themselves.

This standardization also solves another problem: If a developer has set up an application to use a custom TCP protocol, any servers or clients that want to communicate with that application will need to use the same custom protocol.

Additionally, because WebSocket does not need to reestablish connections and send new request and response headers back and forth, it removes a great deal of the overhead of an HTTP connection. This results in a significant improvement in the speed of the connection. That being said, don't forget that the establishment of a WebSocket connection *is a step that affects performance*. If you're not sending messages back and forth fairly rapidly, an HTTP connection is still preferred to avoid that initial overhead.

WebSocket's standardized protocol is supported by all modern browsers and servers, which makes it easy to connect your app to other servers and clients as needed, without having to go through the configuration process all over again.

Conclusion

TCP, HTTP, and WebSocket are connection protocols that you need to know to be an effective web application developer. We've done a good amount of HTTP work so far (especially in our **REST APIs** module)- for more experience with WebSocket, take some time to explore **Socket.IO (https://socket.io/)**, a library that supports simple WebSocket connections from both the client and server sides.

Resources

- **What are IP & TCP? - CloudFlare (https://www.cloudflare.com/learning/ddos/glossary/tcp-ip/)**
- **HTTP - Mozilla (https://developer.mozilla.org/en-US/docs/Web/HTTP)**
- **The WebSocket API (WebSockets) - Mozilla (https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API)**